SUBJECT SPECIFIC PROJECT

TIC TAC TOE GAME

Submitted in partial fulfilment of the requirements of the Course on

Basic Programming Skills

SUBMITTED BY :- RAHUL KUMAR BHADRA

REGD NO:- 2301298467

BRANCH:- CSE

HOUSE:- VISWESWARAY

Name of the Guide

(Prof) Mrs. Rajeswari Chaini

CERTIFICATE

This is to certify that the project titled "Tic Tac Toe“ has been successfully completed by Rahul Kumar Bhadra Regd No. 2301298467s in partial fulfilment of the requirements for the award of the degree of B.Tech in CSE at Gandhi Institute For Technology (Autonomous). This project work has been completed under the guidance of Mrs. Rajeswari Chaini for the academic year 2023 (1st Semester).

Signature of Guide

ACKNOWLEDGEMENT

I express my gratitude to my project guide, **Mrs. Rajeswari Chaini And Mr. Antaryami Muduli** , for their invaluable support and guidance in completing the Tic Tac Toe project.
Thanks to the faculty at **Gandhi Institute For Technology (Autonomous)** for their insights. I'm grateful for the collaborative spirit of my peers and the unwavering support of my family. Each contribution, whether direct or indirect, has been crucial. Thank you all for making this project a success.

Rahul Kumar Bhadra

TABLE OF CONTENTS

SL. No	Contents	Page No
1	Abstract	1
2	Introduction	2
3	Objectives Of The Project	3
4	Features of Tic-Tac-Toe	4
5	Source Code Of The Project	5 – 18
6	Output	19
7	Future Enhancement	20
8	Conclusion	21
9	References	22

ABSTRACT

The square block (3 x 3) can be filled with a cross (X) or a circle (O) in the Tic-Tac-Toe game, which can be played by two players (O). By allowing each player to mark their move, the game will switch between the players. When one of the players makes a horizontal, vertical, or diagonal line with three identical markers, the program displays which player has won, whether X or O. The game is constructed such that two players can play tic tac-toe by entering the position to place their marks on the board. The software will have a display function and a select function for placing the symbol as well as a toggle function for switching between the symbols, allowing each participant to take a turn playing the game. After each player makes a move, the computer will update and check for game circumstances as the game progresses.

KEYWORDS: - Tic-Tac-Toe, Virtual Game, Toggle Turns, Display Winner, etc.

INTRODUCTION

People began to show interest in using their computers as a source of entertainment in the 1970s, resulting in the emergence of a multibillion-dollar game business. The origin is uncertain, though ancient Egyptians claim to have discovered the Tic-Tac-Toe pattern etched on rocks around 3500 years ago. Later, they discovered how much fun it was to play a game using this pattern. The game became popular after that, and it could be played on a wooden board, a table, or even a piece of paper. Tic-Tac Toe is a popular board game in which two players alternately place X's and O's on a 3x3 grid. The players must first pick who would go first and who will record his actions with an X. The game continues with the opponents placing their marks on any unoccupied cell. The goal of the game is for the first player to collect three marks in a row, which might be vertical, horizontal, or diagonal. If all of the cells are filled, the game ends in a tie. Using the C programming language, we created a two-person tic tac toe game in which the player who forms a horizontal, vertical, or diagonal sequence of three marks wins. When a player is successful in forming a vertical, diagonal or horizontal sequence of either X or 0, the game declares the winner. The system has been designed in such a way that the 3X3 grid will be displayed first along with the positions on each square. The first player will then have to enter the position in which he/she wants to place the mark in. The turns are toggled until any one of the player is successful in making a sequence of 3 consecutive marks either vertically, horizontally or diagonally. And finally the winner is declared.

Objective of the Project

The primary objective of undertaking a Tic Tac Toe project in the C programming language is to reinforce and apply foundational programming concepts in a practical setting. By implementing the game, developers aim to:

1. Practice Control Structures: Utilize loops and conditionals to manage the flow of the game, allowing for turns, checking for win conditions, and handling ties.

2. Implement Functions: Break down the program into modular functions, such as those for displaying the game board, accepting player input, checking for a winner, and managing the overall game loop.

3. Work with Arrays: Represent the game board as a 2D array, practicing array manipulation for tracking player moves and determining the state of the game.

4. User Interaction: Develop skills in taking user input, validating it, and updating the game state accordingly. This involves handling input errors and ensuring a user-friendly experience.

5. Logical Problem Solving: Implement the logic for checking win conditions, ensuring that the program accurately identifies when a player has achieved three in a row either horizontally, vertically, or diagonally.

Features of Tic Tac Toe

A Tic Tac Toe project typically includes:

1. Game Board: Display a 3x3 grid for players to make their moves.

2. Players: Allow two players to take turns, often represented as 'X' and 'O'.

3. Input Handling: Receive input from players to place their symbol on the board.

4. Winning Conditions: Check for a winning combination (three in a row horizontally, vertically, or diagonally).

5. Draw Condition: Detect when the game ends in a draw.

6. Turn Management: Keep track of whose turn it is during the game.

7. Game State: Maintain the state of the game (ongoing, won, or drawn).

8. Restart Option: Allow players to restart the game after it concludes.

9. User Interface: Provide a simple interface to interact with the game.

10. Error Handling: Manage invalid inputs and prevent illegal moves.

SOURCE CODE OF THE PROJECT

```
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define COMPUTER 1
#define HUMAN 2
#define SIDE 3
#define COMPUTERMOVE 'O'
#define HUMANMOVE 'X'

 struct Move {

    int row, col;
};

 char player = 'x', opponent = 'o';

bool isMovesLeft(char board[3][3])
{

    for (int i = 0; i < 3; i++)

        for (int j = 0; j < 3; j++)

            if (board[i][j] == '_')

                return true;
 return false;
}

int evaluate(char b[3][3])
{
```

```
    for (int row = 0; row < 3; row++) {

        if (b[row][0] == b[row][1]

            && b[row][1] == b[row][2]) {

            if (b[row][0] == player)

                return +10;

            else if (b[row][0] == opponent)

                return -10;

        }  }

    for (int col = 0; col < 3; col++) {

        if (b[0][col] == b[1][col]

            && b[1][col] == b[2][col]) {

            if (b[0][col] == player)
return +10;

            else if (b[0][col] == opponent)

                return -10;

}}

    if (b[0][0] == b[1][1] && b[1][1] == b[2][2]) {

        if (b[0][0] == player)
```

```
            return +10;
        else if (b[0][0] == opponent)
            return -10;
    }
   if (b[0][2] == b[1][1] && b[1][1] == b[2][0]) {
        if (b[0][2] == player)
            return +10;
        else if (b[0][2] == opponent)
            return -10;
    }
    return 0;
}

int minimax(char board[3][3], int depth, bool isMax)
{
    int score = evaluate(board);
    if (score == 10)
        return score;
    if (score == -10)
```

```
        return score;

    if (isMovesLeft(board) == false)

        return 0;

    if (isMax) {

        int best = -1000;

        for (int i = 0; i < 3; i++) {

            for (int j = 0; j < 3; j++) {

                if (board[i][j] == '_') {

                    board[i][j] = player;

                    int val

                        = minimax(board, depth + 1, !isMax);

                    if (val > best) {

                        best = val; }

board[i][j] = '_';
} }  }
return best;

    }
else {

        int best = 1000;
```

```
        for (int i = 0; i < 3; i++) {

            for (int j = 0; j < 3; j++) {

                if (board[i][j] == '_') {

                    board[i][j] = opponent;

                    int val

                        = minimax(board, depth + 1, !isMax);

                    if (val < best) {

                        best = val;   }

                    board[i][j] = '_';

}}}
        return best;

    }}

struct Move findBestMove(char board[3][3])
{

    int bestVal = -1000;

    struct Move bestMove;

    bestMove.row = -1;

    bestMove.col = -1;
```

```
    for (int i = 0; i < 3; i++) {

        for (int j = 0; j < 3; j++) {

if (board[i][j] == '_') {

                board[i][j] = player;

                int moveVal = minimax(board, 0, false);

                board[i][j] = '_';

                if (moveVal > bestVal) {

                    bestMove.row = i;

                    bestMove.col = j;

                    bestVal = moveVal;

                } } } }

  return bestMove;
}



void showBoard(char board[][SIDE])
{

    printf("\n\n");

    printf("\t\t\t %c | %c | %c \n", board[0][0],

          board[0][1], board[0][2]);
```

```
    printf("\t\t\t--------------\n");

    printf("\t\t\t %c | %c | %c \n", board[1][0],

          board[1][1], board[1][2]);

    printf("\t\t\t--------------\n");

    printf("\t\t\t %c | %c | %c \n\n", board[2][0],

          board[2][1], board[2][2]);
}

 // Function to show the instructions

void showInstructions()
{

    printf("\t\t\t Tic-Tac-Toe\n\n");

    printf("Choose a cell numbered from 1 to 9 as below "

          "and play\n\n");



    printf("\t\t\t 1 | 2 | 3 \n");

    printf("\t\t\t--------------\n");

    printf("\t\t\t 4 | 5 | 6 \n");

    printf("\t\t\t--------------\n");
```

```
    printf("\t\t\t 7 | 8 | 9 \n\n");

    printf("-\t-\t-\t-\t-\t-\t-\t-\t-\n\n");
}

void initialise(char board[][SIDE], int moves[])
{

    srand(time(NULL));

    for (int i = 0; i < SIDE; i++) {

        for (int j = 0; j < SIDE; j++)

            board[i][j] = ' ';

    }
    for (int i = 0; i < SIDE * SIDE; i++)

        moves[i] = i;

    for (int i = 0; i < SIDE * SIDE; i++) {

        int randIndex = rand() % (SIDE * SIDE);

        int temp = moves[i];

        moves[i] = moves[randIndex];

        moves[randIndex] = temp;

    }  }
void declareWinner(int whoseTurn)
{
 if (whoseTurn == COMPUTER)
```

```
        printf("COMPUTER has won\n");

    else

        printf("HUMAN has won\n");
}

int rowCrossed(char board[][SIDE])
{

    for (int i = 0; i < SIDE; i++) {

        if (board[i][0] == board[i][1]

            && board[i][1] == board[i][2]

            && board[i][0] != ' ')

            return 1;

    }

    return 0;
}

int columnCrossed(char board[][SIDE])
{
 for (int i = 0; i < SIDE; i++) {

        if (board[0][i] == board[1][i]

            && board[1][i] == board[2][i]

            && board[0][i] != ' ')
```

```
            return 1;  }
    return 0;  }
int diagonalCrossed(char board[][SIDE])
{

    if ((board[0][0] == board[1][1]

         && board[1][1] == board[2][2]

         && board[0][0] != ' ')

        || (board[0][2] == board[1][1]

            && board[1][1] == board[2][0]

            && board[0][2] != ' '))

        return 1;

  return 0;
}
int gameOver(char board[][SIDE])
{

    return (rowCrossed(board) || columnCrossed(board)

            || diagonalCrossed(board));
}
void playTicTacToe(int whoseTurn)
{

    char board[SIDE][SIDE];

    int moves[SIDE * SIDE];
```

```
    initialise(board, moves);
    showInstructions();


int moveIndex = 0, x, y;

    while (!gameOver(board) && moveIndex != SIDE * SIDE) {

        if (whoseTurn == COMPUTER) {

        char tempBoard[3][3];

            for (int i = 0; i < 3; i++) {

                for (int j = 0; j < 3; j++) {

                    if (board[i][j] == 'X') {

                        tempBoard[i][j] = 'x';

                    }

                    else if (board[i][j] == 'O') {

                        tempBoard[i][j] = 'o';

                    }

                    else {

                        tempBoard[i][j] = '_';

                    }  }  }
```

```
        struct Move thisMove = findBestMove(tempBoard);
        x = thisMove.row;
        y = thisMove.col;

board[x][y] = COMPUTERMOVE;
        printf("COMPUTER has put a %c in cell %d %d\n",
              COMPUTERMOVE, x, y);
        showBoard(board);
        moveIndex++;
        whoseTurn = HUMAN;  }
      else if (whoseTurn == HUMAN) {
        int move;
        printf("Enter your move (1-9): ");
        scanf("%d", &move);
        if (move < 1 || move > 9) {
          printf("Invalid input! Please enter a "
                "number between 1 and 9.\n");
          continue;
```

```
        }
        x = (move - 1) / SIDE;
        y = (move - 1) % SIDE;
        if (board[x][y] == ' ') {
            board[x][y] = HUMANMOVE;
            showBoard(board);
            moveIndex++;
            if (gameOver(board)) {
                declareWinner(HUMAN);
                return;  }
            whoseTurn = COMPUTER;
        }
        else {
            printf("Cell %d is already occupied. Try "
                "again.\n",
                move);
        }  }  }
if (!gameOver(board) && moveIndex == SIDE * SIDE)
```

```
        printf("It's a draw\n");

    else {


if (whoseTurn == COMPUTER)

            whoseTurn = HUMAN;

        else if (whoseTurn == HUMAN)

            whoseTurn = COMPUTER;

declareWinner(whoseTurn);

    }   }


int main()
{
playTicTacToe(COMPUTER);

  return 0;
}
```

Output

```
COMPUTER has put a O in cell 0 0

                 O |   |
              --------------
                 |   |
              --------------
                 |   |

Enter your move (1-9):
```

```
Enter your move (1-9): 5

                 O |   |
              --------------
                 | X |
              --------------
                 |   |

COMPUTER has put a O in cell 0 1

                 O | O |
              --------------
                 | X |
              --------------
                 |   |

Enter your move (1-9):
```

```
Enter your move (1-9): 3

                 O | O | X
              --------------
                 | X |
              --------------
                 |   |

COMPUTER has put a O in cell 1 0

                 O | O | X
              --------------
                 O | X |
              --------------
                 |   |

Enter your move (1-9):
```

```
COMPUTER has put a O in cell 1 0

                 O | O | X
              --------------
                 O | X |
              --------------
                 |   |

Enter your move (1-9): 7

                 O | O | X
              --------------
                 O | X |
              --------------
                 X |   |

HUMAN has won
```

Future Enhancement

Instead of displaying the output on the console, an attractive GUI can be added to the game with a login and signup system for storing previous data. The game can also be equipped with different levels such as easy, medium and difficult

Conclusion

We have built a Tic-Tac-Toe using the C programming language. Tic Tac Toe is a game that is played by people of all ages. It has been developed using the C programming language. Any purpose-driven decision maker can have intelligence as a trait. A Tic Tac Toe algorithm has been presented in the system and tested, and it has proven to be effective. Chiefly, the system is free of errors.

Reference

- Implementation of Tic-Tac-Toe game - GeeksforGeeks
- tic tac toe project - Search Images (bing.com)
- tic-tac-toe-game · GitHub Topics · GitHub